Q-1 Create a React application that fetches data from JSONPlaceholder using the Fetch API and display a list of of users/posts. Implement a loading indicator while data is being fetched.

Components/Loader.jsx

```
 import React from "react";

function Loader() {
  return <h3>Loading data...</h3>;
}

export default Loader;
```

Components/ErrorMessage.jsx

```
import React from "react";

function ErrorMessage({ message }) {
  return <h3 style={{ color: "red" }}>{message}</h3>;
}

export default ErrorMessage;
```

Components/UserList.jsx

```
import React from "react";

function UserList({ users }) {
  return (
    <div>
      <h2>Users List</h2>
      {users.map((user) => (
        <div
          key={user.id}
          style={{
            border: "1px solid #ccc",
            padding: "10px",
            margin: "10px 0",
            borderRadius: "8px",
          }}
        >
          <h3>{user.name}</h3>
          <p>Email: {user.email}</p>
          <p>City: {user.address.city}</p>
        </div>
```

```
    ))}
   </div>
  );
}

export default UserList;
```

Components/PostList.jsx

```
import React from "react";

function PostList({ posts }) {
  return (
    <div>
      <h2>Posts List</h2>
      {posts.slice(0, 10).map((post) => (
        <div
          key={post.id}
          style={{
            border: "1px solid #ccc",
            padding: "10px",
            margin: "10px 0",
            borderRadius: "8px",
          }}
        >
          <h3>{post.title}</h3>
          <p>{post.body}</p>
        </div>
      ))}
    </div>
  );
}

export default PostList;
```

Question1.jsx

```
import React, { useEffect, useState } from "react";
import Loader from "./components/Loader";
import UserList from "./components/UserList";
import PostList from "./components/PostList";

function Question1() {
  const [users, setUsers] = useState([]);
```

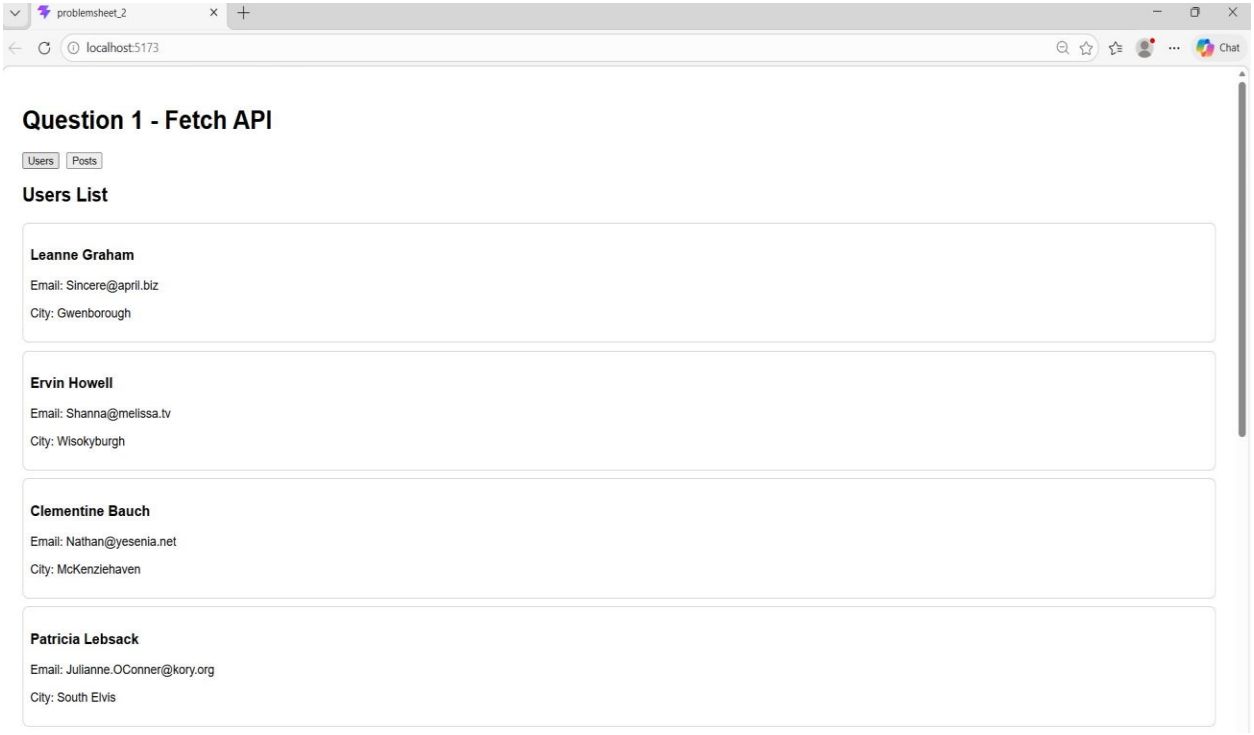
```
const [posts, setPosts] = useState([]);
const [type, setType] = useState("users");
const [loading, setLoading] = useState(false);
const [error, setError] = useState("");

useEffect(() => {
  fetchData("users");
}, []);

const fetchData = async (dataType) => {
  try {
    setLoading(true);
    setError("");

    const url =
      dataType === "users"
        ? "https://jsonplaceholder.typicode.com/users"
        : "https://jsonplaceholder.typicode.com/posts";

    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Failed to fetch data");
    }

    const data = await response.json();

    if (dataType === "users") {
      setUsers(data);
    } else {
      setPosts(data);
    }

    setType(dataType);
  } catch (err) {
    setError("Error fetching data");
  } finally {
    setLoading(false);
  }
};

return (
  <div style={{ padding: "20px", fontFamily: "Arial" }}>
    <h1>Question 1 - Fetch API</h1>
```

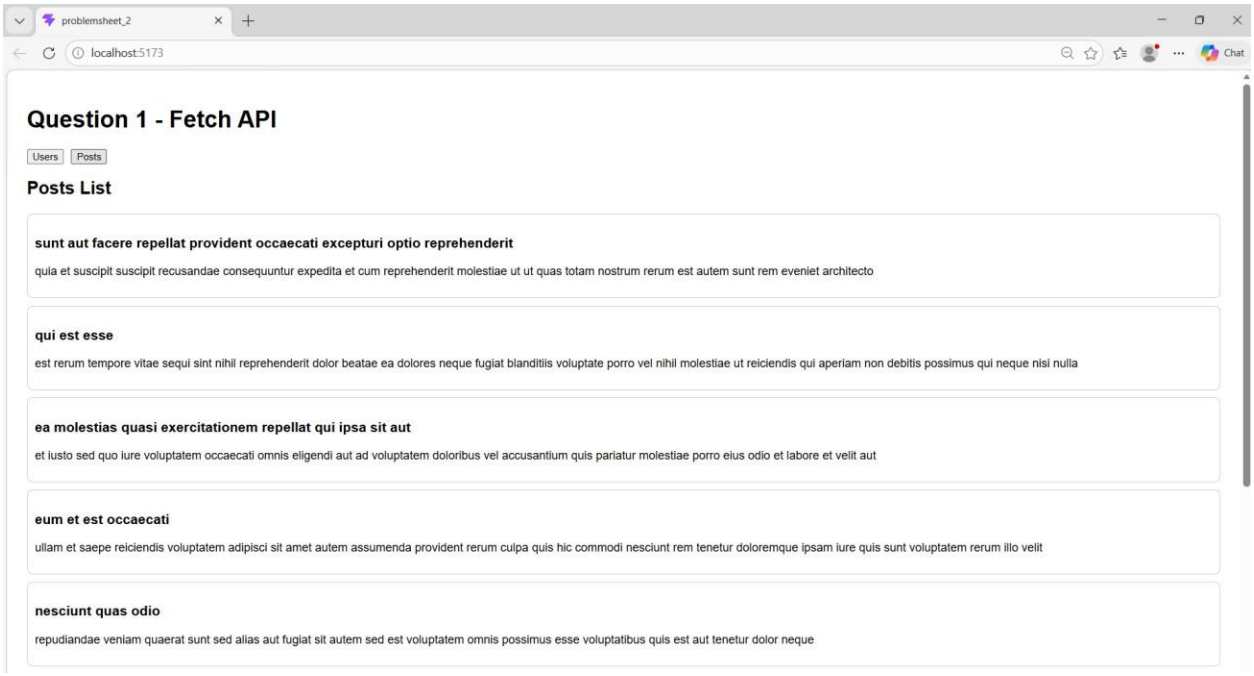
```
      <button onClick={() => fetchData("users")}>Users</button>
      <button onClick={() => fetchData("posts")} style={{ marginLeft: "10px" }}>
        Posts
      </button>

      {loading && <Loader />}
      {error && <h3 style={{ color: "red" }}>{error}</h3>}

      {!loading && !error && type === "users" && (
        <UserList users={users} />
      )}

      {!loading && !error && type === "posts" && (
        <PostList posts={posts} />
      )}
    </div>
  );
}

export default Question1;
```

src/App.jsx

```
import React from "react";
import Question1 from "./Question1";

function App() {
  return <Question1 />;
 }

export default App;
```

OUTPUT :


problemsheet_2
localhost:5173
Chat
Question 1 - Fetch API
Users
Posts
Users List
Leanne Graham
Email: Sincere@april.biz
City: Gwenborough
Ervin Howell
Email: Shanna@melissa.tv
City: Wisokyburgh
Clementine Bauch
Email: Nathan@yesenia.net
City: McKenziehaven
Patricia Lebsack
Email: Julianne.OConner@kory.org
City: South Elvis


problemsheet_2
localhost:5173
Chat
Question 1 - Fetch API
Users
Posts
Posts List
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto
qui est esse
est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla
ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit
nesciunt quas odio
repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

Q-2 Build a React component that uses Axios to fetch data from an API and display it in a structured format (table or cards).

Components/UserCards.jsx

```
import React from "react";

function UserCards({ users }) {
  return (
    <div>
      <h2>User Data</h2>

      <div
        style={{
          display: "grid",
          gridTemplateColumns: "repeat(auto-fit, minmax(250px, 1fr))",
          gap: "15px",
        }}
      >
        {users.map((user) => (
          <div
            key={user.id}
            style={{
              border: "1px solid #ccc",
              borderRadius: "10px",
              padding: "15px",
              boxShadow: "0 2px 5px rgba(0,0,0,0.1)",
              backgroundColor: "#f9f9f9",
            }}
          >
            <h3>{user.name}</h3>
            <p><strong>Email:</strong> {user.email}</p>
            <p><strong>Phone:</strong> {user.phone}</p>
            <p><strong>City:</strong> {user.address.city}</p>
            <p><strong>Company:</strong> {user.company.name}</p>
          </div>
        ))}
      </div>
    </div>
  );
}

export default UserCards;
```

Question-2.jsx

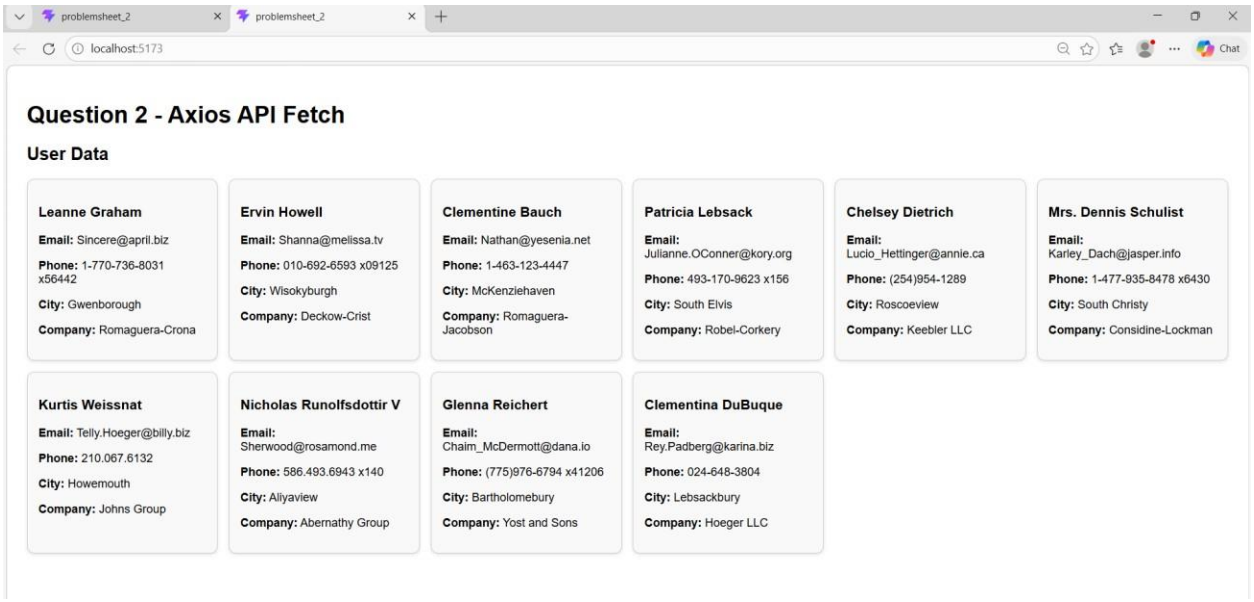
```
import React, { useEffect, useState } from "react";
import axios from "axios";
import Loader from "./components/Loader";
import ErrorMessage from "./components/ErrorMessage";
import UserCards from "./components/UserCards";

function Question2() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    try {
      setLoading(true);
      setError("");

      const response = await axios.get(
        "https://jsonplaceholder.typicode.com/users"
      );

      setUsers(response.data);
    } catch (err) {
      setError("Failed to fetch data");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h1>Question 2 - Axios API Fetch</h1>

      {loading && <Loader />}
      {error && <ErrorMessage message={error} />}
      {!loading && !error && <UserCards users={users} />}
    </div>
  );
}
```

```
export default Question2;
```

src/App.jsx

```
import React from "react";
import Question2 from "./Question2";

function App() {
  return <Question2 />;
}

export default App;
```

OUTPUT :



Q-3 Develop a React application that performs CRUD operations (Create, Read, Update, Delete) using JSONPlaceholder API. Use appropriate HTTP methods (GET,POST,PUT,DELETE).

src/components/Loader.jsx

```
import React from "react";

function Loader() {
  return <h3>Loading data...</h3>;
}

export default Loader;
```

src/components/ErrorMessage.jsx

```
import React from "react";

function ErrorMessage({ message }) {
  return <h3 style={{ color: "red" }}>{message}</h3>;
}

export default ErrorMessage;
```

src/components/PostForm.jsx

```
import React from "react";

function PostForm({
  title,
  body,
  setTitle,
  setBody,
  handleSubmit,
  isEditing,
}) {
  return (
    <div style={{ marginBottom: "20px" }}>
      <h2>{isEditing ? "Update Post" : "Create Post"}</h2>

      <input
        type="text"
        placeholder="Enter title"
        value={title}
```

```
        onChange={(e) => setTitle(e.target.value)}
        style={{
          width: "100%",
          padding: "10px",
          marginBottom: "10px",
          borderRadius: "6px",
          border: "1px solid #ccc",
        }}
      />

      <textarea
        placeholder="Enter body"
        value={body}
        onChange={(e) => setBody(e.target.value)}
        rows="4"
        style={{
          width: "100%",
          padding: "10px",
          marginBottom: "10px",
          borderRadius: "6px",
          border: "1px solid #ccc",
        }}
      />

      <button onClick={handleSubmit}>
        {isEditing ? "Update Post" : "Add Post"}
      </button>
    </div>
  );
}

export default PostForm;
```

src/components/PostListCrud.jsx

```
import React from "react";

function PostListCrud({ posts, handleEdit, handleDelete }) {
  return (
    <div>
      <h2>Posts List</h2>

      {posts.map((post) => (
        <div
```

```
          key={post.id}
          style={{
            border: "1px solid #ccc",
            padding: "10px",
            marginBottom: "10px",
            borderRadius: "8px",
          }}
        >
          <h3>{post.title}</h3>
          <p>{post.body}</p>

          <button onClick={() => handleEdit(post)}>Edit</button>
          <button
            onClick={() => handleDelete(post.id)}
            style={{ marginLeft: "10px" }}
          >
            Delete
          </button>
        </div>
      ))}
    </div>
  );
}

export default PostListCrud;
```

src/Question3.jsx

```
import React, { useEffect, useState } from "react";
import Loader from "./components/Loader";
import ErrorMessage from "./components/ErrorMessage";
import PostForm from "./components/PostForm";
import PostListCrud from "./components/PostListCrud";

function Question3() {
  const [posts, setPosts] = useState([]);
  const [title, setTitle] = useState("");
  const [body, setBody] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [editId, setEditId] = useState(null);

  useEffect(() => {
    fetchPosts();
```

```
}, []);

const fetchPosts = async () => {
  try {
    setLoading(true);
    setError("");

    const response = await fetch(
      "https://jsonplaceholder.typicode.com/posts"
    );

    if (!response.ok) {
      throw new Error("Failed to fetch posts");
    }

    const data = await response.json();
    setPosts(data.slice(0, 10));
  } catch (err) {
    setError("Error fetching posts");
  } finally {
    setLoading(false);
  }
};

const handleSubmit = async () => {
  if (!title || !body) {
    alert("Please fill all fields");
    return;
  }

  if (editId) {
    await updatePost();
  } else {
    await createPost();
  }
};

const createPost = async () => {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/posts",
      {
        method: "POST",
        headers: {
```

```
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          title,
          body,
          userId: 1,
        }),
      }
    );

    if (!response.ok) {
      throw new Error("Failed to create post");
    }

    const newPost = await response.json();

    setPosts([{ ...newPost, id: Date.now() }, ...posts]);
    setTitle("");
    setBody("");
  } catch (err) {
    setError("Error creating post");
  }
};

const handleEdit = (post) => {
  setTitle(post.title);
  setBody(post.body);
  setEditId(post.id);
};

const updatePost = async () => {
  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/posts/${editId}`,
      {
        method: "PUT",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          id: editId,
          title,
          body,
          userId: 1,
```

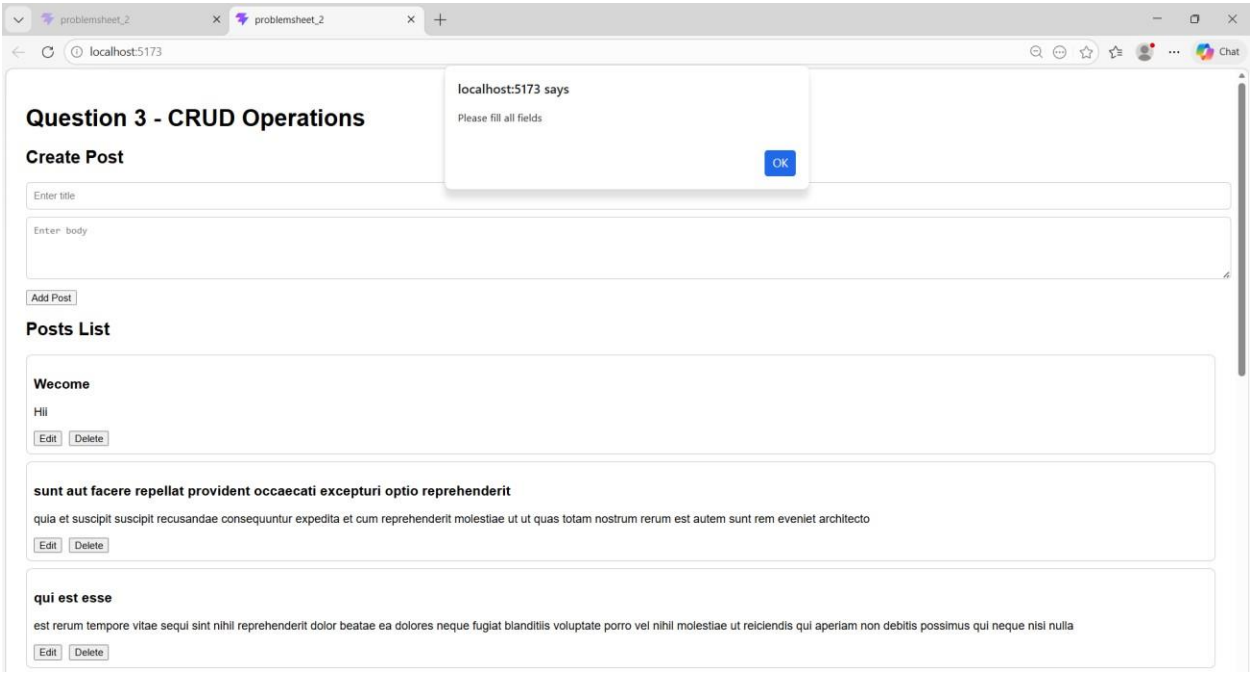
```
      }),
    }
  );

  if (!response.ok) {
    throw new Error("Failed to update post");
  }

  const updatedPost = await response.json();

  const updatedPosts = posts.map((post) =>
    post.id === editId ? { ...post, ...updatedPost } : post
  );

  setPosts(updatedPosts);
  setTitle("");
  setBody("");
  setEditId(null);
 } catch (err) {
  setError("Error updating post");
 }
};

const handleDelete = async (id) => {
 try {
  const response = await fetch(
    `https://jsonplaceholder.typicode.com/posts/${id}`,
    {
      method: "DELETE",
    }
  );

  if (!response.ok) {
    throw new Error("Failed to delete post");
  }

  const filteredPosts = posts.filter((post) => post.id !== id);
  setPosts(filteredPosts);
 } catch (err) {
  setError("Error deleting post");
 }
};

return (
```

```
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h1>Question 3 - CRUD Operations</h1>

      <PostForm
        title={title}
        body={body}
        setTitle={setTitle}
        setBody={setBody}
        handleSubmit={handleSubmit}
        isEditing={editId !== null}
      />

      {loading && <Loader />}
      {error && <ErrorMessage message={error} />}

      {!loading && !error && (
        <PostListCrud
          posts={posts}
          handleEdit={handleEdit}
          handleDelete={handleDelete}
        />
      )}
    </div>
  );
}

export default Question3;
```

src/App.jsx

```
import React from "react";
import Question3 from "./Question3";

function App() {
  return <Question3 />;
}

export default App;
```

OUTPUT :



Q-4 Create a React app that handles API (e.g. network failure, invalid endpoint). Display userfriendly error messages and fallback UI.

src/components/Loader.jsx

```
import React from "react";

function Loader() {
  return <h3>Loading data...</h3>;
}

export default Loader;
```

src/components/ErrorMessage.jsx

```
import React from "react";

function ErrorMessage({ message }) {
  return <h3 style={{ color: "red" }}>{message}</h3>;
}

export default ErrorMessage;
```

src/components/FallbackUI.jsx

```
import React from "react";

function FallbackUI({ onRetry }) {
  return (
    <div
      style={{
        border: "1px solid #f5c2c7",
        backgroundColor: "#f8d7da",
        padding: "20px",
        borderRadius: "8px",
        marginTop: "20px",
      }}
    >
      <h2>Something went wrong</h2>
      <p>We could not load the data. Please try again.</p>
      <button onClick={onRetry}>Retry</button>
    </div>
  );
}

export default FallbackUI;
```

src/Question4.jsx

```
import React, { useState } from "react";
import Loader from "./components/Loader";
import ErrorMessage from "./components/ErrorMessage";
import FallbackUI from "./components/FallbackUI";

function Question4() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [useInvalidUrl, setUseInvalidUrl] = useState(false);

  const fetchUsers = async () => {
    try {
      setLoading(true);
      setError("");
      setUsers([]);

      const url = useInvalidUrl
```

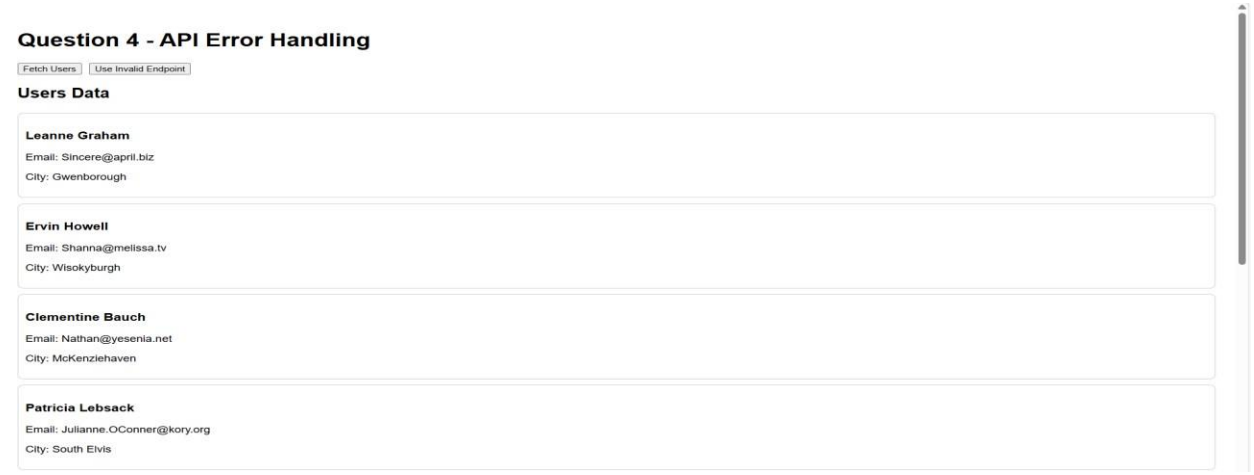
```
      ? "https://jsonplaceholder.typicode.com/invalid-endpoint"
      : "https://jsonplaceholder.typicode.com/users";

    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Invalid API endpoint or server error");
    }

    const data = await response.json();
    setUsers(data);
  } catch (err) {
    if (err.message === "Failed to fetch") {
      setError("Network error. Please check your internet connection.");
    } else {
      setError(err.message || "Something went wrong while fetching data.");
    }
  } finally {
    setLoading(false);
  }
};

return (
  <div style={{ padding: "20px", fontFamily: "Arial" }}>
    <h1>Question 4 - API Error Handling</h1>

    <div style={{ marginBottom: "15px" }}>
      <button onClick={fetchUsers}>Fetch Users</button>

      <button
        onClick={() => setUseInvalidUrl(!useInvalidUrl)}
        style={{ marginLeft: "10px" }}
      >
        {useInvalidUrl ? "Use Valid Endpoint" : "Use Invalid Endpoint"}
      </button>
    </div>

    {loading && <Loader />}

    {error && (
      <>
        <ErrorMessage message={error} />
        <FallbackUI onRetry={fetchUsers} />
      </>
```

```
    )}

    {!loading && !error && users.length > 0 && (
     <div>
      <h2>Users Data</h2>
      {users.map((user) => (
       <div
        key={user.id}
        style={{
         border: "1px solid #ccc",
         padding: "10px",
         marginBottom: "10px",
         borderRadius: "8px",
        }}
       >
        <h3>{user.name}</h3>
        <p>Email: {user.email}</p>
        <p>City: {user.address.city}</p>
       </div>
      ))}
     </div>
    )}

    {!loading && !error && users.length === 0 && (
     <p>No data loaded yet. Click "Fetch Users" to load data.</p>
    )}
   </div>
  );
}

export default Question4;
```

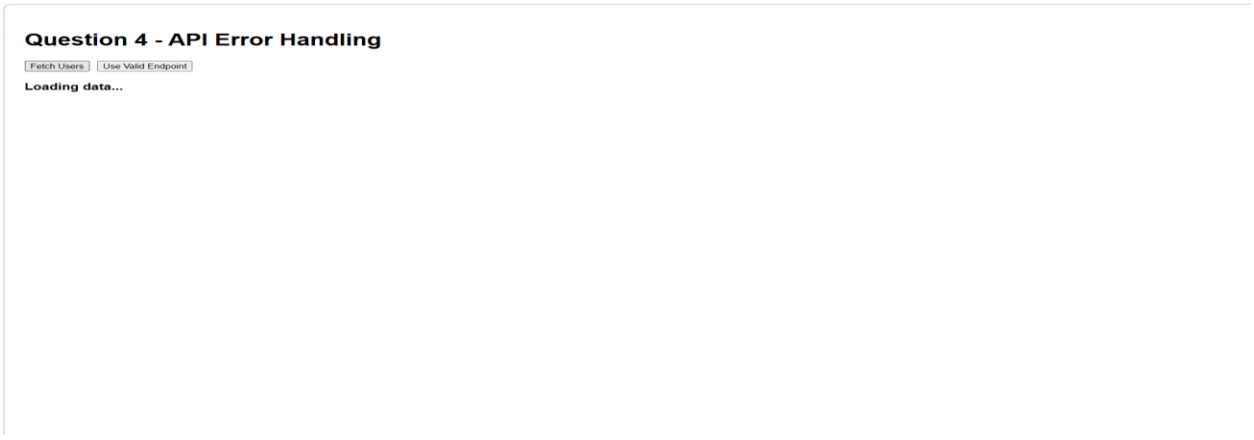
src/App.jsx

```
import React from "react";
import Question4 from "./Question4";

function App() {
  return <Question4 />;
}

export default App;
```

OUTPUT :


Question 4 - API Error Handling
Fetch Users
Use Invalid Endpoint
Users Data
Leanne Graham
Email: Sincere@april.biz
City: Gwenborough
Ervin Howell
Email: Shanna@melissa.tv
City: Wisokyburgh
Clementine Bauch
Email: Nathan@yesenia.net
City: McKenziehaven
Patricia Lebsack
Email: Julianne.OConner@kory.org
City: South Elvis


localhost:5173
Chat
Question 4 - API Error Handling
Fetch Users
Use Valid Endpoint
Invalid API endpoint or server error
Something went wrong
We could not load the data. Please try again.
Retry


Question 4 - API Error Handling
Fetch Users
Use Valid Endpoint
Loading data...

Q-5 Design a component that clearly manages and displays:

- Loading state (spinner/text)
- Success state (data display)
- Error state (error message)

src/components/Loader.jsx

```
import React from "react";

function Loader() {
  return <h3>Loading data...</h3>;
}

export default Loader;
```

src/components/ErrorMessage.jsx

```
import React from "react";

function ErrorMessage({ message }) {
  return <h3 style={{ color: "red" }}>{message}</h3>;
}

export default ErrorMessage;
```

src/components/SuccessData.jsx

```
import React from "react";

function SuccessData({ users }) {
  return (
    <div>
      <h2>Success State - Data Loaded</h2>

      {users.map((user) => (
        <div
          key={user.id}
          style={{
            border: "1px solid #ccc",
            padding: "10px",
            marginBottom: "10px",
            borderRadius: "8px",
          }}
        >
```

```
        <h3>{user.name}</h3>
        <p>Email: {user.email}</p>
        <p>City: {user.address.city}</p>
       </div>
     ))}
   </div>
  );
}

export default SuccessData;
```

src/Question5.jsx

```
import React, { useState } from "react";
import Loader from "./components/Loader";
import ErrorMessage from "./components/ErrorMessage";
import SuccessData from "./components/SuccessData";

function Question5() {
  const [users, setUsers] = useState([]);
  const [status, setStatus] = useState("idle");
  const [error, setError] = useState("");

  const fetchUsers = async () => {
    try {
      setStatus("loading");
      setError("");
      setUsers([]);

      const response = await fetch("https://jsonplaceholder.typicode.com/users");

      if (!response.ok) {
        throw new Error("Failed to fetch data");
      }

      const data = await response.json();
      setUsers(data);
      setStatus("success");
    } catch (err) {
      setError("Error fetching data");
      setStatus("error");
    }
  };
```

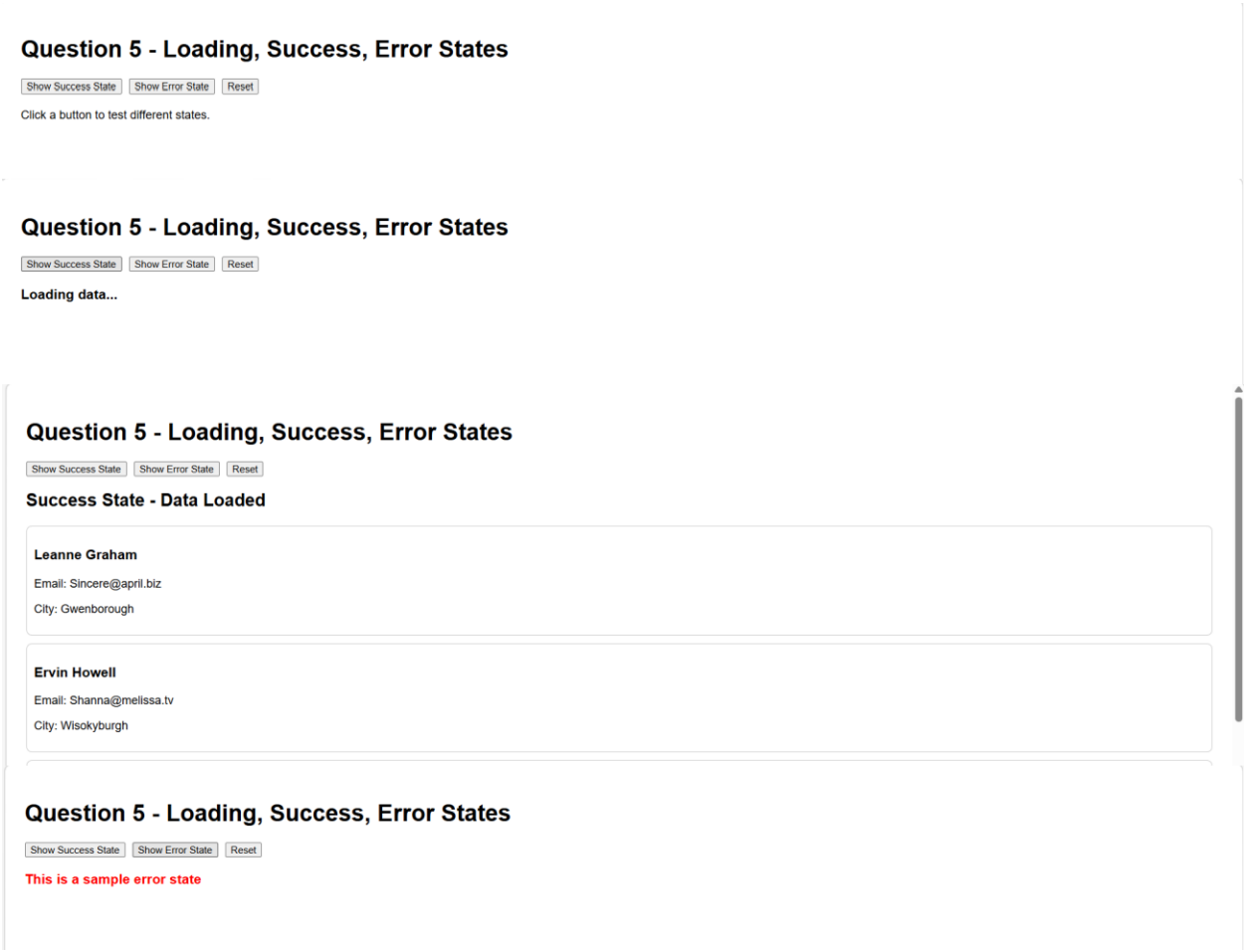
```
  const showErrorState = () => {
    setUsers([]);
    setError("This is a sample error state");
    setStatus("error");
  };

  const resetState = () => {
    setUsers([]);
    setError("");
    setStatus("idle");
  };

  return (
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h1>Question 5 - Loading, Success, Error States</h1>

      <button onClick={fetchUsers}>Show Success State</button>
      <button onClick={showErrorState} style={{ marginLeft: "10px" }}>
        Show Error State
      </button>
      <button onClick={resetState} style={{ marginLeft: "10px" }}>
        Reset
      </button>

      <div style={{ marginTop: "20px" }}>
        {status === "idle" && <p>Click a button to test different states.</p>}
        {status === "loading" && <Loader />}
        {status === "error" && <ErrorMessage message={error} />}
        {status === "success" && <SuccessData users={users} />}
      </div>
    </div>
  );
}

export default Question5;
```

src/App.jsx

```
import React from "react";
import Question5 from "./Question5";

function App() {
  return <Question5 />;
}
```

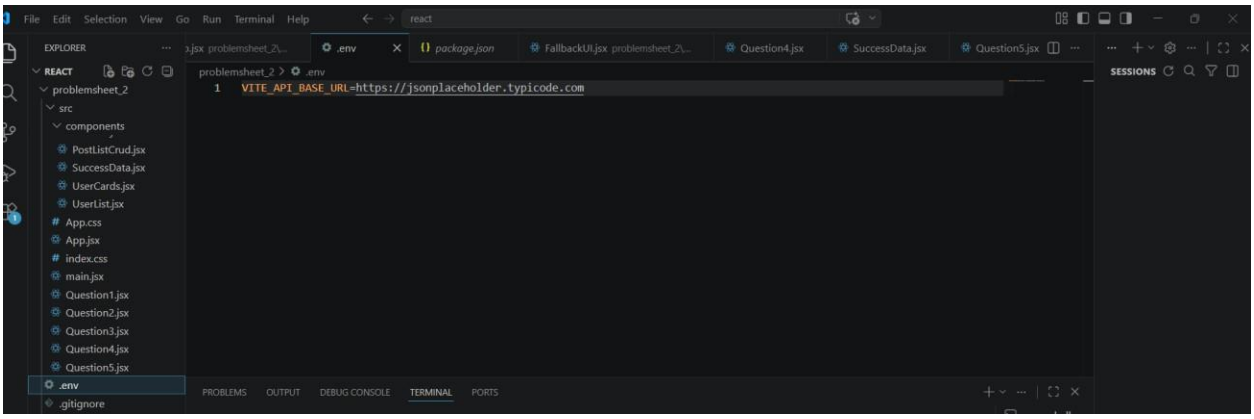
export default App;

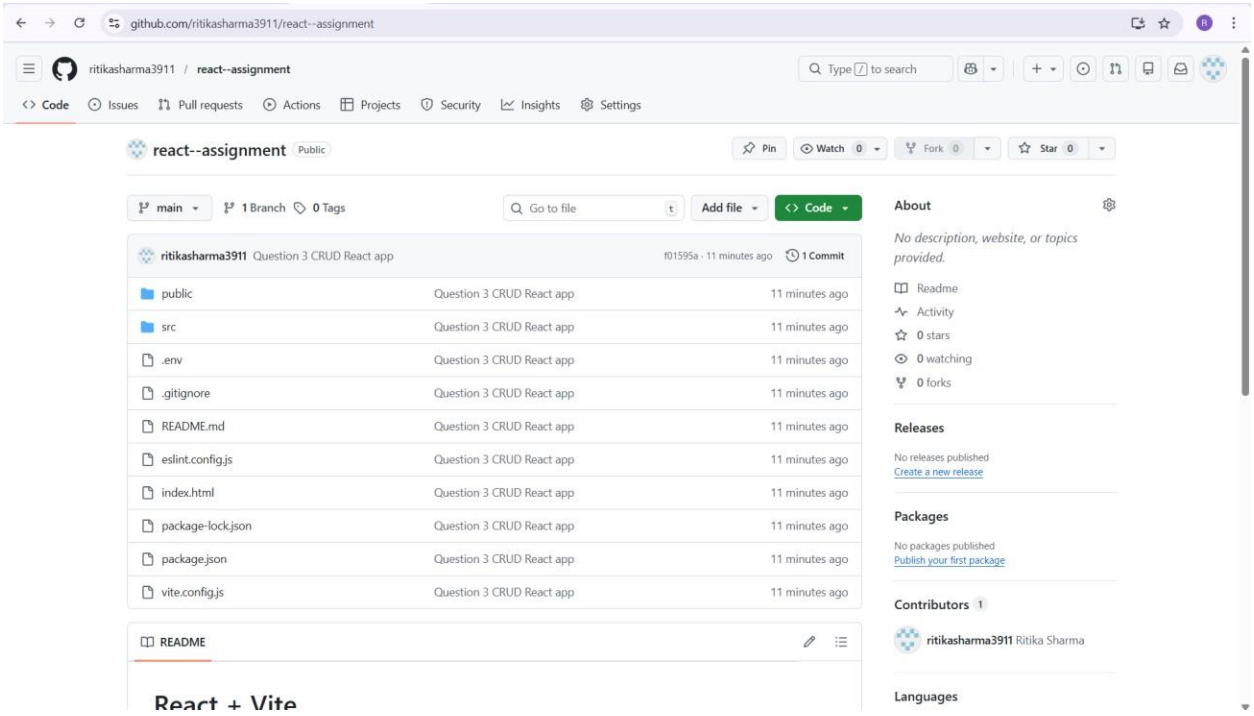
OUTPUT :



Q-6 Deploy any of the above React application like Vercel or Netlify.
Include the following :

- Use of environment variables (.env files)
- GitHub respository link and hosted app link





Git Link :

https://github.com/ajaysonavane/react--assignment.git

Vercel Link

react-assignmentt-orcin.vercel.app